

Universidade Federal do Rio Grande do Norte - UFRN
Departamento de Informática e Matemática Aplicada - DIMAP

Relatório Técnico: Compilador MiniJava

Pré-processador, Analisador Léxico, Analisador Sintático e Tabela de Símbolos

DIM0164 — Compiladores

Professor: Valdicleis S. Costa
Natal, maio de 2026

1. Introdução

Este relatório documenta o desenvolvimento de um compilador para MiniJava, um subconjunto reduzido da linguagem Java amplamente utilizado em disciplinas introdutórias de compiladores. O compilador foi implementado em C++17 e é composto por quatro módulos em sequência: pré-processador, analisador léxico, analisador sintático descendente recursivo e tabela de símbolos.

O objetivo principal do trabalho é realizar as fases iniciais de compilação — da leitura do código-fonte até a verificação sintática completa e o registro dos símbolos declarados. As seções a seguir detalham as estruturas de dados utilizadas em cada componente, as técnicas empregadas na análise sintática e o fluxo completo de execução do sistema.

2. Desenvolvimento

2.1 Estruturas de Dados Utilizadas

Token

A estrutura central do analisador léxico é o Token, definido como um registro com cinco campos. O campo tipo armazena um valor do enum TipoToken, que possui 35 variantes cobrindo palavras-chave, operadores, separadores, identificadores, literais numéricos e tokens especiais (EOF, ERRO). O campo valor guarda a cadeia de caracteres literal do token (por exemplo, "class", "42" ou "myVar"). O campo subtipo é uma string descritiva legível por humanos da categoria do token ("KEYWORD", "NUMBER", "OP", etc.), útil nas mensagens de diagnóstico. Os campos linha e coluna registram a posição exata do token no arquivo original, permitindo que erros sejam reportados com precisão.

```
struct Token { TipoToken tipo; std::string valor; std::string subtipo; int
linha; int coluna; };
```

Essa estrutura permite que os módulos posteriores — o parser, em particular — tomem decisões baseadas tanto no tipo semântico do token quanto em sua posição, viabilizando mensagens de erro no formato "L5:C12 esperado ;".

Symbol

A tabela de símbolos opera sobre registros do tipo Symbol, igualmente uma struct com cinco campos: name (nome do identificador), type (tipo estático, como "int", "boolean", "int[]" ou um nome de classe), category (um dos quatro papéis possíveis: CLASS, METHOD, VARIABLE ou PARAMETER), scope (identificador do escopo de declaração) e line (linha de declaração no arquivo original).

```
struct Symbol { std::string name, type, category, scope; int line; };
```

A escolha de uma struct plana, sem ponteiros ou referências cruzadas, simplifica a implementação sem comprometer a capacidade de representar o escopo: em vez de uma pilha de tabelas aninhadas, o escopo é codificado diretamente como uma string no padrão "ClassName.methodName". Assim, a busca e a verificação de redeclarações reduzem-se a comparações lineares sobre um único vetor.

SymbolTable

A classe SymbolTable encapsula um std::vector<Symbol> e expõe quatro operações: addSymbol (insere um símbolo após verificar duplicatas no mesmo escopo), isDeclared (verifica conflito por comparação exata de name e scope), lookup (busca um símbolo pelo nome) e printTable (exibe a tabela formatada no terminal). A decisão de usar um vetor sequencial — em vez de, por exemplo, um std::unordered_map aninhado — reflete a escala dos programas MiniJava, onde o número de símbolos é pequeno e o custo de busca linear é irrelevante na prática.

Parser

O parser não possui uma estrutura de dados própria além do vetor de tokens recebido do léxico e dos contadores de posição. O estado relevante durante o parsing é mantido em dois campos de instância: `currentClass` (nome da classe sendo analisada no momento) e `currentMethod` (nome do método corrente, ou vazia quando fora de qualquer método). Esses campos são usados para compor o escopo correto ao registrar símbolos na tabela.

2.2 Técnicas Adotadas na Análise Sintática

Eliminação de recursão à esquerda e hierarquia de precedência

A gramática original de expressões em MiniJava é ambígua e apresenta recursão à esquerda, o que impossibilita a aplicação direta de um parser descendente recursivo. A solução adotada foi transformar a gramática por eliminação de recursão à esquerda e por codificação da precedência em camadas de funções mutuamente recursivas. Cada nível de precedência corresponde a exatamente uma função de parsing:

```
parseExpression() → parseAndExp() → parseCompExp() →  
parseAddExp() → parseMulExp() → parseUnaryExp() →  
parsePrimaryExp() → parsePostfixExp()
```

Operadores de menor precedência (&&) são tratados nos níveis superiores da cadeia; os de maior precedência (acesso a array, chamada de método) são resolvidos em `parsePostfixExp`. Esse padrão, chamado de recursão indireta em cadeia, garante associatividade e precedência corretas sem qualquer mecanismo externo de prioridade.

A tabela abaixo resume o esquema de precedência implementado:

Nível	Função	Operadores / Construtos
1 (menor)	<code>parseAndExp</code>	&& (zero ou mais)
2	<code>parseCompExp</code>	> e < (zero ou um)
3	<code>parseAddExp</code>	+ e - (zero ou mais)
4	<code>parseMulExp</code>	* (zero ou mais)
5	<code>parseUnaryExp</code>	! (prefixo unário)
6	<code>parsePrimaryExp</code>	literais, new, this, (Exp)
7 (maior)	<code>parsePostfixExp</code>	[Exp], .length, .método(args)

Lookahead de um token e backtracking conservador

O parser usa lookahead de exatamente um token (LL(1) na prática), exceto em um ponto delicado: a distinção entre declaração de variável e comando de atribuição dentro de um bloco. Quando o token corrente é um identificador e o próximo também é um identificador, trata-se de uma declaração do tipo "T var "; caso contrário, é uma atribuição. Essa decisão é feita em `parseVarDecl` por inspeção de `peek(1)`, sem retrocesso no fluxo de tokens:

```
if (current() == ID && peek() == ID) → declaração de variável
if (current() == ID && peek() == ASSIGN) → atribuição
```

A separação é importante porque variáveis devem ser registradas na tabela de símbolos, enquanto atribuições são apenas validadas sintaticamente.

Recuperação de erros

A estratégia de recuperação adotada é conservadora: ao encontrar um token inesperado, a função `expect` registra uma mensagem de erro com linha e coluna e retorna `false`, mas o parsing continua para o próximo ponto de sincronização. Isso permite que múltiplos erros sejam coletados em uma única passagem. A verificação final conta o número de erros acumulados; se for maior que zero, o compilador reporta todos e encerra com código de saída 1.

2.3 Pré-processador

O pré-processador implementa uma máquina de estados linear que consome o código-fonte caractere a caractere. São reconhecidos dois tipos de comentário: de linha (`//`) e de bloco (`/* */`). Ao encontrar um comentário de linha, o scanner avança até o próximo newline, substituindo toda a sequência por um único espaço — o newline em si é descartado, mas um espaço é inserido para garantir que tokens de linhas adjacentes não se colem. Comentários de bloco são substituídos por um espaço em branco simples. Espaços e tabulações consecutivos são colapsados em um único espaço, e quebras de linha são preservadas quando não forem consecutivas.

A preservação correta de newlines é essencial para o rastreamento de posição no léxico: o analisador léxico incrementa seu contador de linhas a cada `'\n'` encontrado no stream processado, de modo que qualquer supressão indevida de quebras de linha distorceria as posições reportadas nas mensagens de erro.

2.4 Analisador Léxico

O analisador léxico (classe `Lexer`) opera sobre a string já pré-processada. Ele mantém um ponteiro de posição e dois contadores (linha e coluna), atualizados a cada caractere consumido. A tokenização segue uma ordem de prioridade: primeiro verifica sequências alfanuméricas (identificadores e palavras-chave), depois dígitos, depois o operador composto `&&`, depois operadores simples (mapeados em um `std::map<char, TipoToken>`), e por fim separadores (igualmente mapeados).

Caracteres não reconhecidos por nenhuma das categorias geram um token `TOKEN_ERRO` com mensagem de diagnóstico.

O conjunto de 21 palavras-chave é armazenado em um `std::set<std::string>` estático. Após consumir uma sequência alfanumérica, o léxico verifica se o lexema está nesse conjunto; caso positivo, a função `kwTipo` retorna o `TipoToken` específico da palavra-chave; caso negativo, o token é classificado como `ID`. Esse mecanismo evita cadeias de `if-else` em cascata e permite futuras extensões com custo $O(\log n)$.

2.5 Tabela de Símbolos

A tabela de símbolos é preenchida durante o parsing, e não em uma passagem separada. Isso é possível porque `MiniJava` exige declaração antes do uso (exceto para chamadas de método entre classes), tornando desnecessária uma segunda passagem de resolução de nomes. O escopo é representado como concatenação de strings segundo o esquema abaixo:

Categoria	Escopo armazenado	Exemplo
Classe	"GLOBAL"	"Programa" → GLOBAL
Método	"NomeClasse"	"main" → Programa
Variável local / Parâmetro	"NomeClasse.NomeMetodo"	"x" → Programa.main

A detecção de redeclaração compara nome e scope: dois símbolos com o mesmo nome em escopos distintos são permitidos (como é comum em herança e sobrecarga em Java), mas dois símbolos com nome e escopo idênticos são bloqueados — o segundo é silenciosamente descartado e um aviso poderia ser emitido em versões futuras com análise semântica.

A saída da tabela é formatada em um grid de cinco colunas (Nome, Tipo, Categoria, Escopo, Linha) utilizando `std::setw` da biblioteca `<iomanip>`, produzindo um alinhamento legível no terminal sem dependências externas.

2.6 Fluxo de Execução

O ponto de entrada (`main.cpp`) orquestra as quatro etapas em sequência. Primeiro, o arquivo é lido integralmente em memória como uma `std::string`. Em seguida, a função `preprocessar` transforma essa string removendo comentários e normalizando espaços. A string resultante é entregue ao construtor do `Lexer`, que produz um `std::vector<Token>`. Se erros léxicos forem encontrados, o compilador os exibe e encerra. Caso contrário, o vetor de tokens é passado ao `Parser`, que valida a estrutura sintática e popula a tabela de símbolos. Ao final, se não houver erros sintáticos, a tabela é impressa.

Etapas	Entrada	Saída
--------	---------	-------

Leitura	Caminho do arquivo	std::string (código bruto)
Pré-processamento	std::string (código bruto)	std::string (código limpo)
Análise Léxica	std::string (código limpo)	std::vector<Token>
Análise Sintática	std::vector<Token>	Tabela de Símbolos + erros

2.7 Complexidade Computacional

O pré-processador percorre o código-fonte uma única vez, sendo $O(n)$ em relação ao número de caracteres. O léxico também faz uma passagem linear, com custo proporcional ao número de caracteres e ao comprimento médio dos lexemas. O parser percorre o vetor de tokens uma vez de forma determinística — sem retrocesso — produzindo uma derivação em $O(n)$ em relação ao número de tokens. A tabela de símbolos realiza buscas lineares para verificar redeclarações, com custo $O(k)$ por inserção e $O(k^2)$ total para k símbolos, o que é aceitável para programas MiniJava de pequeno a médio porte.

3. Conclusão

O compilador MiniJava implementado cobre com êxito as três primeiras fases do processo de compilação. O pré-processador elimina ruído léxico preservando informação de posição. O analisador léxico reconhece corretamente os 35 tipos de token da linguagem, incluindo o operador de comparação menor-que (<), cuja ausência foi identificada e corrigida durante o desenvolvimento. O parser descendente recursivo valida a estrutura sintática completa de programas MiniJava — incluindo herança, múltiplas classes, controle de fluxo e expressões com precedência correta — e popula a tabela de símbolos com informação de tipo, categoria e escopo.

Como extensões naturais para etapas futuras, destacam-se a análise semântica (verificação de tipos e resolução de referências), a geração de código intermediário e eventuais otimizações. A arquitetura modular adotada, com separação clara entre pré-processador, léxico, parser e tabela de símbolos, facilita a incorporação dessas etapas sem reescrita significativa do código existente.

Alunos:

Carlos Alberto de Lima Neto

Claudivan Costa de Lima Júnior

Osvaldo Heitor de Andrade Tavares de Souza

Luiz Guilherme Carvalho Viana